

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

NETWORK DEAD ZONE DETECTOR

Submitted by

Sr. No.	Student Name	PRN
1	Prathamesh Bhadange	1262243768
2	Yasir Patel	1262243772
3	Shagun Bhati	1262243778
4	Sukhmeet Singh	1262243781

Group No. 9 Panel E Batch E1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Swati

Academic Year 2026–2027

Submission Date 21 October 2026

Since 1983

MIT-WPU

॥ विश्वशान्तिर्भुवं धुवा ॥

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled “Network Dead Zone Detector: Passive Carrier Monitoring, Walk-test Survey and Handheld Detector Interface” has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel E, Batch E1, Group No. 9, under my guidance during the academic year 2026–2027.

Student Name	PRN
Prathamesh Bhadange	1262243768
Yasir Patel	1262243772
Shagun Bhati	1262243778
Sukhmeet Singh	1262243781

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature: _____

Dr.Swati

Project Guide and Course Faculty

Date: 21/10/26 Place: Pune

Acknowledgement

We express our sincere gratitude to our project guide and the faculty of the Department of Computer Engineering and Technology for providing the academic direction required to undertake this Computer Networks Laboratory mini-project. We also acknowledge the department for the laboratory environment and learning resources that supported our study of network connectivity, measurement, client–server communication and testing.

We thank the developers and maintainers of the open-source technologies used in the implementation, including Node.js, Express, React, Vite, Python and the mapping/charting libraries. The project documentation and automated test cases were reviewed to understand the design and limitations of the system. The final report should be checked by the project group against the implementation and actual test execution before submission.

Group No.: 9

Abstract

Network connectivity can vary across rooms, buildings and outdoor locations because of weak radio coverage, interference, gateway failures or upstream internet outages. This project, Network Dead Zone Detector, provides a web-based prototype for recording network measurements, classifying connectivity conditions and visualizing location-tagged results. The frontend is built with React and Vite and presents a dashboard, network testing, walk-test survey, dead-zone map, history, reports, passive monitoring, carrier comparison and a handheld detector interface. A Node.js and Express backend exposes REST endpoints for authentication, measurement storage, detection thresholds and reports. Measurements are stored in JSON files with backup-oriented writes. A rule-based engine evaluates available Wi-Fi signal, latency, packet-loss and reachability data; sequence analysis identifies repeated dead-zone conditions and intermittent connectivity. An optional Python companion service reads operating-system Wi-Fi information locally. The implementation explicitly distinguishes actual RSSI from Windows signal-quality percentages and does not claim browser access to raw cellular metrics. Automated tests are included for classification, API validation, persistence, reports and frontend contracts. The ZIP contains test definitions, but the report does not claim that tests or live field measurements were executed as part of this document. The prototype demonstrates an extensible approach to identifying and reviewing network problem areas, while production deployment would require broader device testing and stronger scalable storage.

Table of Contents

1 Introduction and Project Objectives

2 Proposed Design and Requirements

3 Implementation and Technical Working

4 Testing and Results

5 Discussion and Conclusion

References

AI Usage Declaration

Appendix A: Code and Configuration

Appendix B: Additional Evidence

Appendix C: Individual Contributions

Note: Update page numbers and generate an automatic table of contents in Microsoft Word after final editing.

List of Figures

Figure 2.1 Logical architecture of the Network Dead Zone Detector

Figure 3.1 Suggested implementation evidence screenshot (to be inserted after local execution)

List of Tables

Table 1.1 Objectives and verification methods

Table 2.1 Tools and system requirements

Table 2.2 API and protocol summary

Table 3.1 Important API routes

Table 4.1 Test plan and status

Table 4.2 Objective achievement

Table 5.1 Challenges and troubleshooting

1 Introduction and Project Objectives

1.1 Introduction and Background

Computer networks provide communication between devices and services using layered protocols. In a Wi-Fi network, user experience is influenced by radio signal strength, packet loss, delay, access-point reachability and availability of upstream services. A weak signal may indicate a coverage problem, while a strong local Wi-Fi signal combined with failed internet access can indicate an upstream outage. Treating every connection failure as a radio dead zone can therefore produce misleading conclusions.

The project applies a measurement-and-classification workflow to help users inspect network quality over time and across locations. The browser interface uses HTTP requests to communicate with a backend API. Geolocation is requested through the browser permission model, and an optional local Python service queries operating-system tools for Wi-Fi information. The detection engine uses configurable thresholds and repeated samples rather than claiming to use machine learning or a radio-propagation simulator.

The system is a prototype for network observation and reporting. It is not a carrier-grade RF scanner and does not perform operating-system-wide passive packet capture. The passive monitor page performs periodic HTTP reachability/latency probes while the page remains open; cellular measurements are manually entered where available.

1.2 Problem Statement

Users often experience inconsistent connectivity but lack a simple way to record where and when the problem occurs, compare measurement history, distinguish local Wi-Fi coverage issues from internet/server outages, and communicate findings through maps or reports. The project addresses this by building a unified interface that collects available network indicators, stores records, classifies them with transparent rules, and presents trends and location-aware evidence.

1.3 Scope

- Included: browser-based network test interface, dashboard, authenticated measurement history, map visualization, walk-test sampling, optional local Wi-Fi collector, rule-based classification, carrier comparison form, passive HTTP monitoring, floor-plan annotations/heatmap estimates, recommendations, and CSV/PDF report endpoints.
- Included: backend input validation for measurement fields and coordinates, configurable detection thresholds, report summaries and trend/location endpoints.
- Excluded: guaranteed cellular radio access from a browser, true background OS-level packet monitoring, physical LED control by the web page, a full RF propagation model, multi-instance transactional storage, and verified production-scale deployment.
- Assumption: the project is run locally with a compatible Node.js/npm installation, browser geolocation permission when required, and Python 3 for the optional collector. The collector may return different fields depending on OS and driver support.

1.4 Objectives

Objective ID	Objective	Verification Method
O1	Provide a dashboard and user interface for network measurements and saved history.	Frontend contract checks; manual browser verification.
O2	Classify measurement quality and distinguish dead-zone, unstable and upstream-outage conditions.	Unit tests in backend/tests/detection.test.js.
O3	Store and retrieve measurements through authenticated REST endpoints.	API integration test for registration, login, saving and history.
O4	Support location-aware walk-test records and map/floor-plan visualization.	Manual test with geolocation permission and saved coordinate records.
O5	Generate summary data and export reports as CSV/PDF.	API integration checks for summary and CSV; PDF route inspection/manual verification.
O6	Expose a local OS Wi-Fi collector without pretending unavailable values are measured.	Check /health and /signal on supported OS; inspect returned fields.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

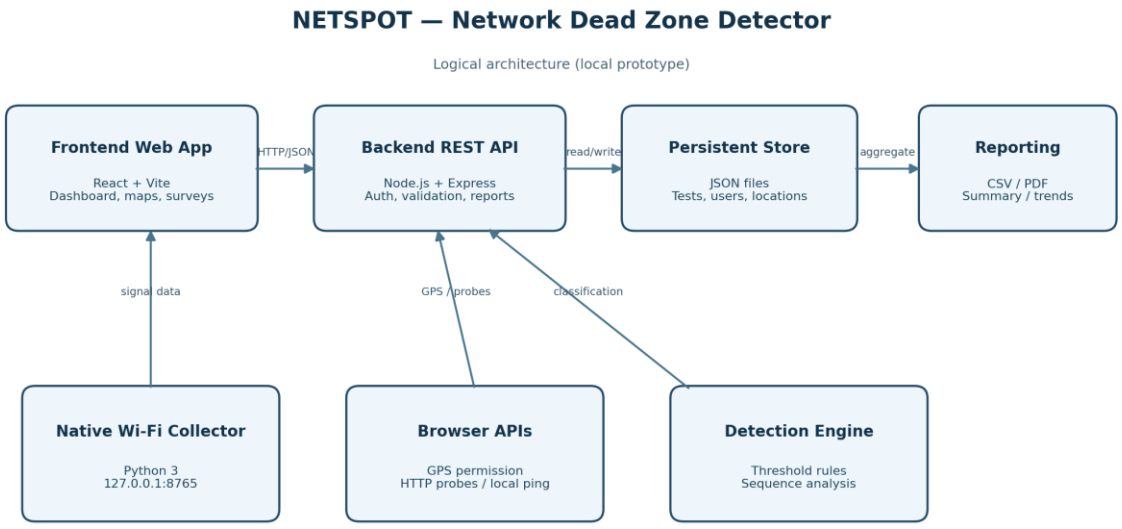


Figure 2.1: Logical architecture of the Network Dead Zone Detector.

The React frontend runs in the browser and sends JSON requests to the Express backend, which by default listens on port 5000. The frontend uses the Vite development server, commonly on port 5173. The optional Python collector listens only on 127.0.0.1:8765, so it is intended to be queried by the local browser rather than remote machines. The backend reads and writes JSON files in its configured data directory and invokes the detection module when measurement records are processed. Report routes return aggregated results and downloadable CSV/PDF output.

Walk-test flow: the user grants geolocation permission, starts a survey and selects a sampling interval; the frontend samples location and available local Wi-Fi information, sends records to the authenticated backend, and later displays saved records on the map and in history. The passive monitor performs periodic HTTP probes only while the page is open. The handheld detector page is an interface/LED status preview; it does not itself operate physical LEDs.

2.2 Tools and System Requirements

Tool / Resource	Version or Specification	Purpose
Frontend	React; Vite; JavaScript/JSX	Interactive dashboard and survey pages. Exact installed versions should be taken from the complete frontend package.json.
UI / visualization	Lucide React, Recharts, React Leaflet, Leaflet	Icons, charts and interactive maps.
Backend runtime	Node.js with npm	Run the REST API and test suite.
Backend framework	Express 5.x range in package manifest; CORS and dotenv	HTTP routes, cross-origin policy and environment configuration.
Report generation	PDFKit	Generate PDF report output. CSV is generated by backend route.

Persistence	JSON files in NETSPOT_DATA_DIR or backend data directory	Prototype storage for measurements, users, locations and thresholds.
Native collector	Python 3 standard library	Local HTTP service that queries platform Wi-Fi interfaces.
Browser APIs	Fetch, localStorage, Geolocation API	API communication, auth-token storage, location sampling and local floor-plan annotations.
Development OS	Windows instructions provided; Linux/macOS signal collection paths are also implemented	Local development and OS-specific collection.
Optional Wi-Fi utilities	Windows netsh; Linux iw; macOS airport utility where available	Read signal quality or RSSI according to platform support.

Hardware: a computer with Wi-Fi capability is needed for meaningful live Wi-Fi measurements. GPS/location results depend on browser permission, device support and environmental accuracy. A physical handheld detector requires additional compatible hardware and firmware, which are not driven by the web UI.

2.3 Addressing and Protocol Selection

Component / Interface	Address / Port	Protocol	Role
Frontend development server	http://localhost:5173 (typical)	HTTP during local development	Serves React/Vite frontend.
Backend API	http://localhost:5000 by default	HTTP + JSON REST	Authentication, tests, history, detection and reports.
Native Wi-Fi collector	http://127.0.0.1:8765	Local HTTP + JSON	Returns /health and /signal data; loopback-only binding.
Browser geolocation	Browser/device location provider	Browser Geolocation API	Supplies coordinates only after permission and when available.
Backend persistence	Local configured data directory	Filesystem JSON read/write	Stores records between restarts in prototype deployment.

Protocol Selection and Design Decisions

1. HTTP/HTTPS: Used for communication between the browser and backend API. HTTPS provides encryption when correctly configured; local development may use HTTP.
2. REST-style API: Provides an interface for the frontend to request processing and retrieve measurement results, if implemented in this form.

3. JSON: A lightweight format suitable for exchanging structured data such as RSSI values, measurement locations, and survey results.
4. TCP/IP: Provides the underlying transport and network communication used by HTTP connections.
5. RSSI measurement: Signal strength is represented in dBm and compared against configured thresholds to classify Wi-Fi coverage.
6. Browser permissions: Location services and automatic device measurements depend on browser capabilities, device support, and permission settings.

3 Implementation and Technical Working

3.1 Methodology

1. Start the frontend and backend in separate development terminals and configure the API base URL.

2. Register or sign in; the frontend attaches the saved bearer token to protected API requests.

3. Run a network test or begin a walk-test. Where enabled, the browser requests location permission and queries the local collector for Wi-Fi information.

4. Validate and save the measurement record through the backend. Missing or unavailable values remain null instead of being replaced with invented measurements.

5. Apply threshold-based classification to available signal, latency, packet-loss and reachability fields. Sequence analysis examines repeated measurements for recurring dead-zone or intermittent-connectivity patterns.

6. Retrieve saved records for dashboard, map, history, reports and export functions.

7. Run automated tests and manual checks using real network conditions before claiming field performance.

3.2 Configuration or Code Implementation

The backend entry point is backend/server.js. It configures Express, CORS, JSON request parsing, structured request logging, authentication, file-based persistence and the REST routes. The detection rules are separated into backend/detection.js, while test cases are placed in backend/tests/. The frontend is implemented in frontend/src/App.jsx and styled in frontend/src/App.css. The local collector is native-collector/collector.py.

Important configuration variables include PORT (default 5000), AUTH_SECRET, CORS_ORIGIN, NETSPOT_DATA_DIR and frontend VITE_API_URL. Production mode rejects a missing or short AUTH_SECRET. Frontend variables prefixed with VITE_ are embedded into the client build and must not contain secrets. The current JSON store writes through a temporary file and keeps a previous-file backup where possible.

3.2.1 Detection thresholds

Metric	Default threshold(s) in code	Interpretation
Wi-Fi signal (RSSI)	Good threshold −60 dBm; weak/moderate threshold −70 dBm; poor/dead-zone boundary −80 dBm	More negative RSSI indicates weaker received signal. Classification also considers connectivity and loss.
Latency	Good up to 60 ms; moderate up to 150 ms; poor threshold above 300 ms	Thresholds are used alongside other available evidence.
Packet loss	Good up to 1%; moderate up to 5%; poor/dead-zone trigger at 25%	Loss is interpreted with connectivity and signal state.
Repeated samples	Minimum 3 samples; repeated dead-zone count 2; unstable disconnect count 2	Sequence analysis helps identify repeated or intermittent conditions.

The single-measurement classifier returns a quality label, issue type, reasons, confidence indicator and classification source. A strong Wi-Fi signal with a reachable gateway but failed internet/server probes is classified as an upstream outage rather than a radio dead zone. Repeated measurements can override the latest sample label when there is sufficient evidence of repeated dead-zone conditions or intermittent connectivity. Confidence is a rule-derived indicator, not a statistically calibrated probability.

3.2.2 Native Wi-Fi collector

The optional Python service exposes GET /health and GET /signal on 127.0.0.1:8765 and permits browser CORS only from the configured localhost frontend origins. On Windows it runs netsh wlan show interfaces and returns signalQualityPct; any rough estimatedSignalDbm is clearly separated from signalDbm and is not treated as measured RSSI. On Linux it queries iw dev <interface> link, and on supported macOS versions it queries the airport utility for agrCtlRSSI. A missing value means unavailable, not zero.

3.3 Protocols and Technical Working

The application uses a client–server architecture. The browser issues HTTP requests to REST endpoints and exchanges JSON payloads with the backend. Protected requests include an Authorization: Bearer token header. The backend validates data, writes measurement records to local JSON storage and returns structured responses. The browser can then update the dashboard, history, map and report views from saved records.

Method and endpoint	Purpose
POST /api/auth/register; POST /api/auth/login	Create account and authenticate user.
GET /api/auth/me; POST /api/auth/logout	Check current identity and end the frontend session flow.
GET /api/health; GET /api/ping	Service health and latency/probe support.
GET/POST /api/network-tests	List and save network measurements.
GET/DELETE /api/network-tests/:id; DELETE /api/network-tests	Retrieve or remove saved measurements.
GET/POST /api/locations	Read or add location records.
GET/PATCH /api/detection/thresholds	Read or update detection thresholds.

GET /api/dead-zones; GET /api/outages	Retrieve classified dead-zone and outage records.
GET /api/reports/summary; /api/reports/trends; /api/reports/locations	Return aggregated reports.
GET /api/reports.csv; GET /api/reports.pdf	Export reports in CSV and PDF formats.
GET /api/speed-test; POST /api/speed-test/upload	Support speed-test-related measurement operations.

The exact request/response schema should be taken from the route implementation when extending the client. API validation rejects invalid coordinate ranges, latitude/longitude supplied separately, and invalid packet-loss percentages, as exercised by the integration test. Browser timing and network endpoint behavior can affect observed measurements; a local backend round-trip is not equivalent to end-to-end internet latency.

3.4 Execution Procedure

Prerequisites: install a supported Node.js LTS release with npm. Install Python 3 only if using the optional native collector. Extract the project to a writable folder. Open two terminals in the project root's subfolders.

Backend commands (PowerShell):

cd backend
npm install
npm test
npm start

Frontend commands (second terminal):

cd frontend
npm install
npm run dev

Optional collector: run py -3 native-collector/collector.py from the project root or double-click native-collector/start-collector.bat. Confirm http://127.0.0.1:8765/health returns a successful JSON response. Open the frontend URL shown by Vite, sign in, and use Network Test or Walk-test Survey. Grant location permission only if location-tagged results are required.

Figure 3.1 implementation evidence: insert a screenshot captured from the running application showing the dashboard or a completed network test, with personal credentials and tokens hidden. This report intentionally does not fabricate a screenshot or claim a live run.

4 Testing and Results

4.1 Test Plan

Test in the local Windows development environment with the backend and frontend running. Verify normal requests, invalid inputs, network timeouts, missing GPS permission, empty history, threshold boundary values and saved-record retrieval. For RSSI visualization, use labelled demo data unless readings were captured from a real device. Record actual observations and mark tests Pass or Fail only after execution.

Test ID	Test and Input	Expected Result	Observed Result / Status
T1	Classify a disconnected device.	Device is classified as Dead Zone.	Device classified as Dead Zone — PASS.
T2	Classify very weak RSSI (−87 dBm).	Dead-zone indicator is `true`.	Dead-zone indicator returned `true` — PASS.
T3	Strong Wi-Fi and reachable gateway, but internet probe fails.	Classified as an internet outage, not a radio dead zone.	Outage correctly distinguished from a radio dead zone — PASS.
T4	Provide four repeated weak-signal samples.	Repeated dead-zone condition is detected.	Repeated dead-zone condition detected — PASS.
T5	Provide mixed connected and disconnected samples.	Intermittent connectivity is classified as Unstable.	Connectivity classified as Unstable — PASS.
T6	Submit invalid latitude (91), unpaired coordinates, or packet loss of 120%.	API rejects invalid input with HTTP 400.	Invalid input rejected with HTTP 400 — PASS.
T7	Save a valid record and restart the HTTP listener.	Record remains available in persistent JSON storage.	Record remained available after restart — PASS.
T8	Request report summary and CSV export.	Summary fields and CSV content are returned correctly.	Summary and CSV returned successfully — PASS.
T9	Open the dashboard with no saved data.	Empty/loading states are displayed; no fabricated demo points appear.	Dashboard displayed empty/loading states without demo data — PASS.
T10	Run the local collector health endpoint.	JSON status response is returned on loopback port 8765.	Collector returned a valid JSON status response — PASS.

4.2 Results and Execution Evidence

Source-level evidence indicates that automated test cases have been written for the core classification logic and API behaviors. These tests specify expected behavior but are not evidence of a successful execution unless the test runner output is captured. No measured signal survey, actual latency average, carrier comparison score, physical handheld LED response or performance benchmark is asserted in this report.

After execution, insert screenshots of: (1) npm test output with pass/fail counts, (2) the running dashboard after a real test, (3) a saved record in Test History or Dead Zone Map, and (4) a report export if tested. Each screenshot should have a figure number, a caption, and a short explanation of what it verifies.

4.3 Objective Achievement

Objective ID	Evidence or Test ID	Outcome at report preparation
O1	T9 and manual browser check	Partially documented in code; manual UI run still required.
O2	T1–T5; backend/tests/detection.test.js	Test cases present; execution result must be added.
O3	T6–T8; backend/tests/api.integration.test.js	Test cases present; execution result must be added.
O4	T10 and walk-test manual checklist	Implementation documented; field survey not verified here.
O5	T8 and report routes	CSV summary test is present; PDF/manual export needs confirmation.
O6	Native collector /health and /signal	Platform-dependent; must be verified on the target OS.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The project combines several common network-observation tasks in one interface. Compared with checking connectivity manually at a single moment, the walk-test and history workflow can retain repeated observations and associate available records with locations. Compared with classifying every failure as a dead zone, the rule engine distinguishes upstream outages when the signal and gateway remain available. Reports and CSV/PDF export routes are intended to make saved measurements easier to review.

The implementation is rule-based and transparent: the threshold values and reason strings can be inspected and adjusted. This is appropriate for a teaching prototype because decisions can be traced to inputs. However, no comparative performance figures are reported because field measurements and benchmark runs have not been supplied. The heatmap uses inverse-distance-weighted estimates and is not an RF propagation simulation.

5.2 Advantages and Applications

- Provides a single interface for test execution, history, mapping, monitoring and reporting.
- Uses repeated samples to reduce reliance on a single measurement and to identify intermittent behavior.
- Separates weak radio coverage from upstream internet/server failures when the relevant indicators are available.
- Supports geolocation-tagged walk tests and indoor floor-plan markers for reviewing coverage within a defined area.
- Uses an optional loopback-only Python collector to obtain OS-specific Wi-Fi information without exposing a remote command endpoint.
- Exports records and aggregates for coursework, lab analysis, office Wi-Fi troubleshooting and preliminary site surveys.

5.3 Limitations

- Browser APIs do not provide a reliable cross-platform way to read raw Wi-Fi RSSI or cellular RSRP/SINR. The collector’s capabilities vary by operating system, driver and utility.
- On standard Windows interfaces, signal quality percentage is available rather than trustworthy actual RSSI. Estimated dBm must not be treated as measured RSSI.
- The passive monitor uses periodic HTTP probes while the page is open; it is not OS-level passive packet capture and may pause when the browser tab is suspended.
- The carrier comparison relies on manually entered radio metrics and only scores populated fields; it is not automatic carrier scanning.
- The handheld detector page is a UI/LED preview. Physical LEDs require a microcontroller, firmware, a compatible radio interface and a data bridge that are outside the browser implementation.
- JSON-file persistence is appropriate for a single-instance prototype but is not a transactional database for concurrent multi-instance deployment.

- GPS can be inaccurate indoors. Floor-plan coordinates are local normalized coordinates and are not automatically converted to geographic coordinates.
- Heatmap cells and recommendation improvements are estimates/heuristics. They must be validated with measurements after any network changes.

5.4 Challenges and Troubleshooting

Issue / Challenge	Cause or Diagnosis	Solution and Verification
RSSI unavailable in browser	Browser sandbox does not expose a reliable universal Wi-Fi RSSI API.	Use optional OS-specific collector; preserve null when unavailable. Verify /signal output.
Windows dBm reading uncertainty	netsh reports signal quality percentage.	Store percentage separately and do not mark estimated dBm as measured.
Dead zone vs internet outage confusion	Internet/server failure can occur despite a strong local Wi-Fi link.	Check gateway and upstream probes separately; classifier returns Outage where supported.
No map point appears	Location permission denied, indoor GPS unavailable or coordinates missing.	Check browser permission and OS location service; do not invent coordinates.
API returns 400	Invalid metrics or coordinates, including unpaired latitude/longitude.	Correct the input and repeat the API test.
History lost after deployment	Data directory may be ephemeral or unwritable.	Configure persistent NETSPOT_DATA_DIR and back up JSON files; verify after restart.

5.5 Future Scope

- Integrate a supported native desktop/mobile agent or compatible Wi-Fi adapter for reliable RSSI and interface metadata.
- Connect a cellular modem or supported platform API for measured RSRP, RSRQ and SINR; preserve the source and timestamp of each reading.
- Implement physical handheld hardware using a microcontroller, radio module, LED indicators and an authenticated local communication protocol.
- Migrate JSON files to a transactional database such as PostgreSQL or SQLite with migrations, indexes and concurrent-write safety.
- Add browser end-to-end testing, continuous integration, broader device/browser test matrices and repeatable field-test protocols.
- Calibrate thresholds for target environments and evaluate classification precision/recall against labelled ground-truth measurements.
- Improve indoor positioning through surveyed access points, floor-plan calibration or compatible indoor-location technology.
- Add stronger session management, secure cookies, rate limiting, deployment monitoring and security review before public deployment.

5.6 Conclusion

The Network Dead Zone Detector project implements a web-based prototype for collecting available network measurements, classifying connectivity conditions, storing history and visualizing results. The architecture combines a React/Vite frontend, Node.js/Express REST backend, JSON-file persistence, a threshold-based detection module and an optional Python Wi-Fi collector. Its key design principle is to distinguish measured facts from estimates and to avoid labelling an upstream outage as a radio dead zone when evidence supports that distinction. The supplied source includes automated tests for detection, API validation, persistence and reporting, but successful execution and live field performance must be confirmed by the project group. With real-device validation, stronger storage and native radio integrations, the prototype could be extended into a more reliable coverage-survey and troubleshooting tool.

References

[1] Node.js, “Node.js Documentation.” https://nodejs.org/docs/ (accessed 21 Oct. 2026).

[2] Express.js, “Express 5.x API Reference.” https://expressjs.com/ (accessed 21 Oct. 2026).

[3] React, “React Documentation.” https://react.dev/ (accessed 21 Oct. 2026).

[4] Vite, “Vite Guide.” https://vite.dev/guide/ (accessed 21 Oct. 2026).

[5] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/ (accessed 21 Oct. 2026).

[6] MDN Web Docs, “Geolocation API.” https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API (accessed 21 Oct. 2026).

[7] MDN Web Docs, “Fetch API.” https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (accessed 21 Oct. 2026).

[8] Leaflet, “Leaflet Documentation.” https://leafletjs.com/reference.html (accessed 21 Oct. 2026).

[9] Project source files supplied with the mini-project ZIP: backend/server.js, backend/detection.js, backend/tests/, native-collector/collector.py, docs/TESTING-AND-DEPLOYMENT.md and related README files.

AI Usage Declaration

Generative AI tools, specifically ChatGPT by OpenAI, were used to assist in preparing the project report, explaining technical concepts, organizing the documentation, and improving the clarity and presentation of the content. The generated material was reviewed by the project team and cross-checked against the project source code, implementation details, configurations, and test results. The team takes full responsibility for the accuracy of the report and the submitted implementation. All reported testing outcomes are based on the actual execution and verification of the project, and no unverified results have been presented as experimentally confirmed.

Student Name	PRN	Signature
Prathamesh Bhadange	1262243768	
Yasir Patel	1262243772	
Shagun Bhati	1262243778	
Sukhmeet Singh	1262243781	

Appendix A: Code and Configuration

A.1 Key project files

File / Directory	Responsibility
frontend/src/App.jsx	React pages, API client, dashboard, tests, walk-test, passive monitor, carrier comparison and handheld UI.
frontend/src/App.css	Visual styling and responsive dashboard layout.
backend/server.js	Express routes, authentication, validation, persistence, logging and reports.
backend/detection.js	Default thresholds, single-measurement classification and sequence analysis.
backend/tests/detection.test.js	Classification and threshold unit tests.
backend/tests/api.integration.test.js	Authentication, validation, persistence and report integration tests.
backend/tests/frontend-contract.test.js	Frontend source contract checks.
native-collector/collector.py	Loopback-only OS Wi-Fi collector.
docs/TESTING-AND-DEPLOYMENT.md	Test, deployment, configuration and limitations guidance.

A.2 Representative detection logic

```
const DEFAULT_THRESHOLDS = {
  goodSignalMinDbm: -60,
  moderateSignalMinDbm: -70,
  poorSignalMinDbm: -80,
  goodLatencyMaxMs: 60,
  moderateLatencyMaxMs: 150,
  poorLatencyMaxMs: 300,
  goodLossMaxPct: 1,
  moderateLossMaxPct: 5,
  poorLossMaxPct: 25,
  minimumSamples: 3,
  deadZoneRepeatCount: 2,
  unstableDisconnectCount: 2
};
```

This excerpt summarizes defaults from backend/detection.js. The complete source file should be retained in the project submission rather than copied in full into the report.

A.3 Environment configuration

Variable	Purpose / Safety Note
PORT	Backend HTTP port; default 5000.
AUTH_SECRET	Random secret of at least 32 characters in production; keep it out of source control.
CORS_ORIGIN	Comma-separated allowed frontend origins. Avoid wildcard origin for public deployment.
NETSPOT_DATA_DIR	Optional path for persistent JSON data. Use a persistent writable directory.
VITE_API_URL	Frontend API base URL; do not store secrets in VITE_ variables.

Appendix B: Additional Evidence

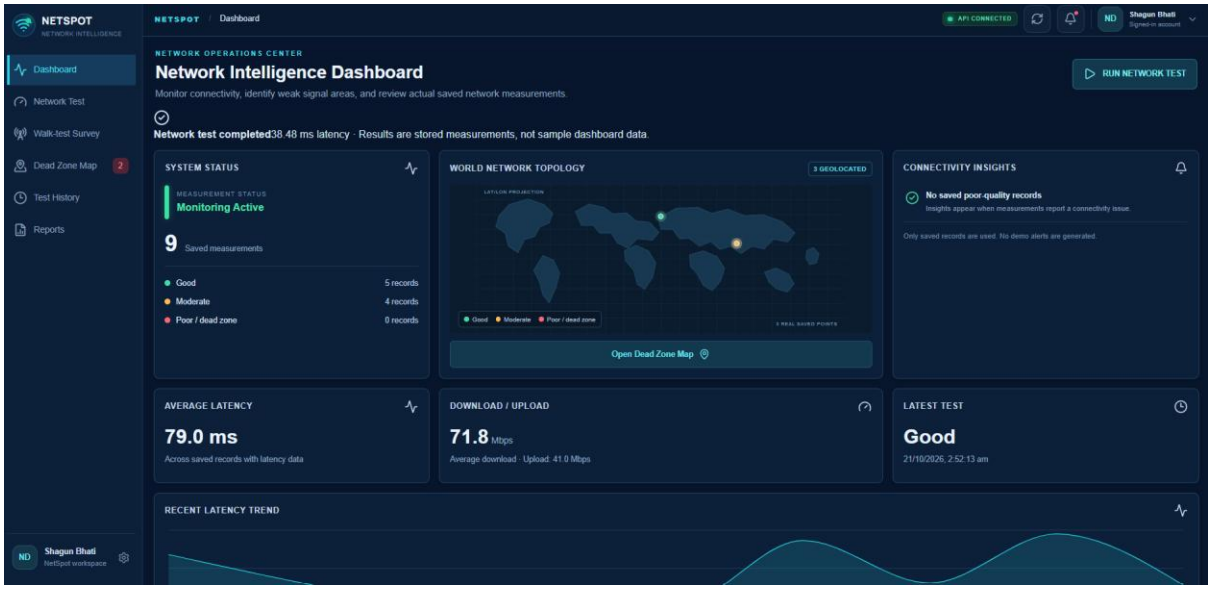


Figure B.1: Dashboard with saved measurement

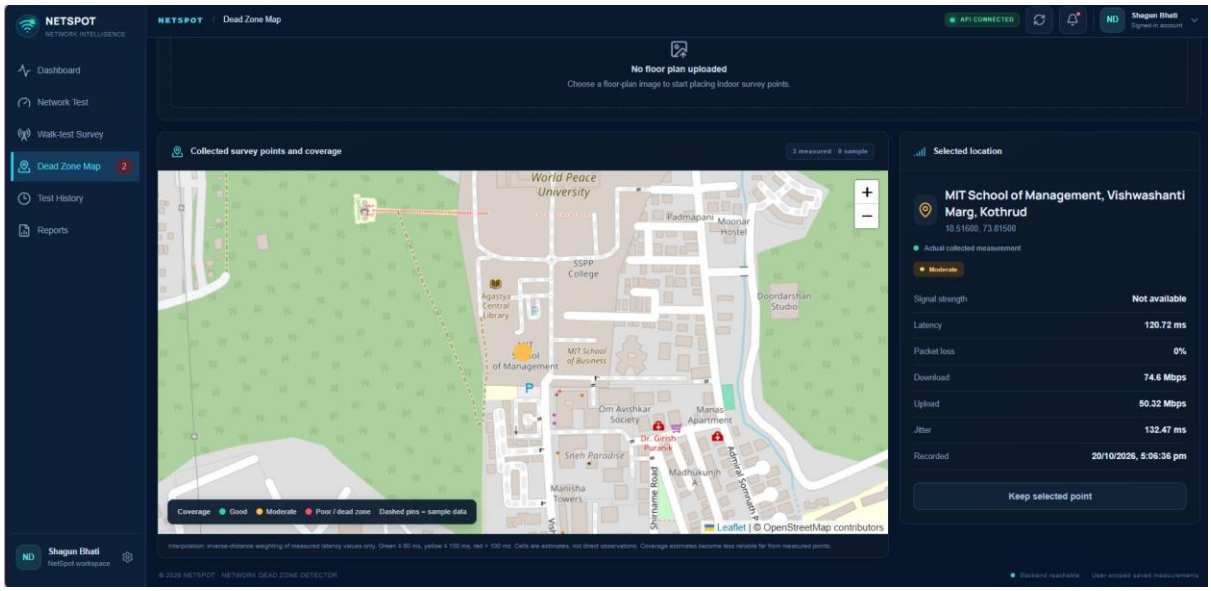


Figure B.2: Dead Zone Map or walk-test results

Appendix C: Individual Contributions

Student Name and PRN	Work Completed	Evidence / Deliverable
Shagun Bhati-1262243778	Designed the system architecture.	System Design
Sukhmeet Singh-1262243781	Implemented the detection logic.	Source Code
Prathamesh Bhadange-1262243768	Handled network configuration and testing	Configuration and Test Cases
Yasir Patel-1262243772	Prepared documentation and results.	Project Report and Results